

# MARAN SUITE SYSTEM — Especificaciones de Desarrollo: Chatbot + Web Check-in

**Fecha:** Marzo 2026

**Módulos:** Integraciones diferenciadoras

**Prioridad:** Alta — Features de diferenciación competitiva

---

## Contexto general

Estas dos features son las que posicionan al sistema por encima de la mayoría de hoteles independientes. El chatbot ya existe como sistema separado; se trata de conectarlo al PMS para que las notificaciones y solicitudes lleguen a las áreas correspondientes. El web check-in es una feature nueva que permite al huésped completar su registro antes de llegar, reduciendo la fricción en recepción.

Ambas features comparten la misma infraestructura de notificaciones internas.

---

## PARTE 1 — Sistema de notificaciones internas (base para ambas features)

**Complejidad:** Media | **Área:** Backend + Frontend

**Debe implementarse primero — es la base de todo lo que sigue.**

### Descripción

Antes de conectar el chatbot o el web check-in, el sistema necesita un mecanismo de notificaciones internas en tiempo real. Cuando el chatbot recibe una solicitud o un huésped completa el web check-in, el sistema debe avisar a las áreas correspondientes sin que tengan que recargar la página.

### Cambios en el schema ( `shared/schema.ts` )

**Nueva tabla** `systemNotifications` :

```
export const systemNotifications = pgTable("system_notifications", {  
  id: serial("id").primaryKey(),
```

```

type: text("type").notNull(),
// Tipos: "web_checkin", "chatbot_request", "chatbot_housekeeping",
//        "chatbot_maintenance", "chatbot_restaurant", "chatbot_spa"
title: text("title").notNull(),
message: text("message").notNull(),
targetArea: text("target_area").notNull(),
// Áreas: "reception", "housekeeping", "maintenance", "restaurant", "spa", "all
relatedEntityType: text("related_entity_type"),
// Ej: "reservation", "guest", "room"
relatedEntityId: integer("related_entity_id"),
isRead: boolean("is_read").default(false),
readAt: timestamp("read_at"),
readBy: text("read_by"),
priority: text("priority").notNull().default("normal"),
// Valores: "low", "normal", "high", "urgent"
createdAt: timestamp("created_at").defaultNow(),
});

```

## Nuevos endpoints ( `server/routes.ts` )

|       |                                                    |                                      |
|-------|----------------------------------------------------|--------------------------------------|
| GET   | <code>/api/notifications?area=reception</code>     | → Listar notificaciones por área (no |
| POST  | <code>/api/notifications</code>                    | → Crear notificación (usado por chat |
| PATCH | <code>/api/notifications/:id/read</code>           | → Marcar como leída                  |
| PATCH | <code>/api/notifications/read-all</code>           | → Marcar todas como leídas (por área |
| GET   | <code>/api/notifications/unread-count?area=</code> | → Cantidad de no leídas por área     |

## Cambios en el frontend

`client/src/components/app-sidebar.tsx`

Agregar un indicador de notificaciones en el sidebar:

- Ícono de campana  en el header del sidebar con badge de cantidad de no leídas.
- Al hacer clic, abre un panel lateral o dropdown con las notificaciones recientes.
- Cada notificación muestra: título, mensaje, área, tiempo transcurrido ("hace 5 min"), prioridad.
- Las urgentes se muestran en rojo, las normales en gris.
- Botón "Marcar todas como leídas".

### Polling o actualización automática:

- Implementar polling cada 30 segundos a `GET /api/notifications/unread-count` para

mantener el badge actualizado sin necesidad de WebSockets.

- Cuando hay notificaciones nuevas, el badge se actualiza automáticamente.

---

## PARTE 2 — Integración del chatbot con el sistema

**Complejidad:** Media | **Área:** Backend

### Descripción

El chatbot ya está en producción y atiende consultas, pedidos y solicitudes de huéspedes. La integración consiste en que cuando el chatbot recibe una solicitud que requiere acción de un área del hotel, envíe una notificación al sistema para que el área correspondiente la vea y actúe.

### Arquitectura de integración

El chatbot se comunica con el PMS mediante una **API webhook**. Cuando el chatbot detecta una solicitud accionable, hace un POST al endpoint del PMS con los datos de la solicitud.

**El PMS no necesita conocer la lógica interna del chatbot** — solo recibe el evento y crea la notificación correspondiente.

### Nuevo endpoint webhook ( `server/routes.ts` )

```
POST /api/webhook/chatbot
```

### Headers requeridos:

```
X-Chatbot-Secret: [clave secreta compartida entre chatbot y PMS]  
Content-Type: application/json
```

### Body esperado:

```
{  
  "eventType": "guest_request",  
  "area": "housekeeping",  
  "priority": "normal",  
  "guestName": "Carlos García",  
  "roomNumber": "302",  
  "reservationId": 45,
```

```
{
  "message": "El huésped solicita toallas adicionales",
  "timestamp": "2026-03-05T14:30:00Z"
}
```

### Valores posibles de `area` :

- `housekeeping` → solicitudes de limpieza, toallas, amenities
- `maintenance` → problemas técnicos (TV, AC, plomería)
- `restaurant` → pedidos de room service
- `spa` → consultas o reservas de tratamientos
- `reception` → solicitudes generales, consultas de factura, late check-out
- `all` → avisos generales

### Lógica del endpoint:

1. Validar el header `X-Chatbot-Secret` .
2. Crear una notificación en `systemNotifications` con los datos recibidos.
3. Si `area = "housekeeping"` y la solicitud implica una tarea (ej: limpieza, toallas), opcionalmente crear una tarea en `housekeepingTasks` automáticamente.
4. Retornar `{ success: true, notificationId: 123 }` .

### Mapeo de solicitudes a áreas

El chatbot debe clasificar las solicitudes antes de enviarlas. Sugerencia de mapeo:

| Tipo de solicitud                 | Área destino | Prioridad |
|-----------------------------------|--------------|-----------|
| Toallas, amenities, almohadas     | housekeeping | normal    |
| Limpieza de habitación            | housekeeping | normal    |
| Problema con TV, AC, luz          | maintenance  | high      |
| Problema de plomería              | maintenance  | urgent    |
| Room service, pedido de comida    | restaurant   | normal    |
| Consulta o reserva de SPA         | spa          | normal    |
| Factura, hora de salida, equipaje | reception    | normal    |

|                   |           |        |
|-------------------|-----------|--------|
| Queja o incidente | reception | urgent |
|-------------------|-----------|--------|

## Dashboard del chatbot (existente)

Si el chatbot ya tiene su propio dashboard, agregar en ese dashboard un enlace o sección "Ver en sistema" que lleve al área correspondiente del PMS con las notificaciones filtradas.

---

## PARTE 3 — Web Check-in

**Complejidad:** Alta | **Área:** Backend + Frontend (página pública externa)

### Descripción

El web check-in permite al huésped completar su registro antes de llegar al hotel, desde su celular mediante un link enviado por WhatsApp. El formulario es una página pública (sin login) identificada por un token único por reserva. Una vez completado, el sistema notifica a recepción y actualiza la reserva con los datos confirmados.

### Flujo completo

1. Recepcionista genera link en el sistema → link con token único
  2. Sistema envía (o el recepcionista copia) el link por WhatsApp al huésped
  3. Huésped abre el link en su celular y completa el formulario
  4. Al enviar, los datos se guardan en el sistema
  5. Recepción recibe notificación y ve los datos confirmados
  6. Recepcionista asigna habitación y confirma el check-in
- 

## 3.1 — Cambios en el schema ( `shared/schema.ts` )

**Nueva tabla** `webCheckins` :

```
export const webCheckins = pgTable("web_checkins", {
  id: serial("id").primaryKey(),
  reservationId: integer("reservation_id").notNull().references(() => reservation),
  token: text("token").notNull().unique(),
  // Token único generado al crear el link (UUID o string aleatorio largo)
  status: text("status").notNull().default("pending"),
  // Estados: "pending" (link enviado, no completado), "completed" (huésped lo co
```

```

// Datos confirmados por el huésped:
confirmedFirstName: text("confirmed_first_name"),
confirmedLastName: text("confirmed_last_name"),
confirmedDocumentType: text("confirmed_document_type"), // DNI, Pasaporte, etc.
confirmedDocumentNumber: text("confirmed_document_number"),
confirmedNationality: text("confirmed_nationality"),
confirmedPhone: text("confirmed_phone"),
confirmedEmail: text("confirmed_email"),

// Foto del documento:
documentPhotoUrl: text("document_photo_url"), // URL o base64 de la imagen subi

// Datos de llegada:
estimatedArrivalTime: text("estimated_arrival_time"), // Hora estimada, ej: "15
requestEarlyCheckIn: boolean("request_early_check_in").default(false),
earlyCheckInTime: text("early_check_in_time"),

// Firma de términos:
termsAccepted: boolean("terms_accepted").default(false),
termsAcceptedAt: timestamp("terms_accepted_at"),
ipAddress: text("ip_address"), // Para registro legal

// Metadata:
completedAt: timestamp("completed_at"),
expiresAt: timestamp("expires_at"), // El link expira 24hs después del check-in
createdAt: timestamp("created_at").defaultNow(),
});

```

---

## 3.2 — Nuevos endpoints ( `server/routes.ts` )

### Endpoints privados (requieren autenticación del sistema):

```

POST /api/web-checkin/generate/:reservationId
→ Genera token y crea registro en webCheckins
→ Retorna el link completo: https://[dominio]/web-checkin/[token]

GET /api/web-checkin/list
→ Lista todos los web check-ins con su estado (para vista en recepción)

GET /api/web-checkin/:reservationId/status
→ Estado del web check-in de una reserva específica

```

### Endpoints públicos (sin autenticación — acceso por token):

GET /api/public/web-checkin/:token

→ Valida el token y retorna los datos de la reserva para pre-llenar el formulario

→ Retorna error si el token expiró o ya fue completado

POST /api/public/web-checkin/:token

→ Recibe y guarda los datos completados por el huésped

→ Actualiza webCheckins con status: "completed"

→ Actualiza la reserva con los datos confirmados (nombre, documento, etc.)

→ Crea notificación en systemNotifications con type: "web\_checkin", targetArea: "

→ Retorna { success: true }

---

### 3.3 — Página pública del formulario

**Nueva ruta en `client/src/App.tsx` :**

`/web-checkin/:token` → `WebCheckinPage` (componente público, sin sidebar ni auth)

`client/src/pages/web-checkin-public.tsx` **(nuevo archivo):**

Página diseñada para mobile, sin el layout del sistema (sin sidebar, sin header interno).

#### Diseño:

- Header con logo del hotel y nombre "Maran Suites & Towers".
- **Indicador de pasos:** [1. Tus datos] → [2. Documento] → [3. Llegada] → [4. Confirmación]

#### Paso 1 — Datos personales:

- Nombre y apellido (pre-llenados desde la reserva, editables).
- Tipo de documento (DNI / Pasaporte / Cédula) + número.
- Nacionalidad.
- Teléfono y email.

#### Paso 2 — Foto del documento:

- Instrucción clara: "Sacá una foto de tu DNI o pasaporte".
- Botón que abre la cámara del celular o permite subir imagen desde galería ( `<input type="file" accept="image/*" capture="environment">` ).
- Preview de la imagen seleccionada.

- La imagen se envía como base64 en el POST.

### **Paso 3 — Tu llegada:**

- Campo "¿A qué hora estimás llegar?" (input time).
- Toggle "¿Necesitás early check-in?" — si activa, campo de hora deseada y texto: "Sujeto a disponibilidad, te confirmaremos a la brevedad."
- Sección "Términos y condiciones" con texto del hotel y checkbox obligatorio "Acepto los términos y condiciones de alojamiento".

### **Paso 4 — Confirmación:**

- Mensaje: "¡Listo! Tu check-in está pre-registrado 🎉"
  - Resumen: nombre, fechas de estadía, hora estimada de llegada.
  - Texto: "En recepción te estaremos esperando. Si tenés alguna consulta, escribinos por WhatsApp."
  - No mostrar número de habitación (aún no está asignada).
- 

## **3.4 — Integración en recepción ( `client/src/pages/check-in.tsx` )**

### **Badge de web check-in en reservas:**

- En la lista de reservas pendientes de check-in, mostrar un badge verde "✓ Web Check-in" para las reservas que tienen `webCheckins.status = "completed"`.
- Esto permite al recepcionista priorizar y agilizar el check-in de quienes ya completaron el formulario.

### **Panel de web check-in completado:**

- Al seleccionar una reserva con web check-in completado, mostrar un panel con los datos confirmados por el huésped, la foto del documento y la hora estimada de llegada.
- Botón "Confirmar datos y asignar habitación" que pre-completa el formulario de check-in con los datos del web check-in.

### **Vista de gestión de web check-ins en recepción:**

- Agregar una tab o sección "Web Check-ins" en la página de Check-in o en el Dashboard.



- Lista de reservas próximas (próximas 48hs) con su estado de web check-in:
    - Sin enviar → botón “Generar y copiar link”
    - Link enviado (pending) → indicador de espera
    - Completado → badge verde con hora de completado
- 

### 3.5 — Envío del link por WhatsApp

El sistema **no envía automáticamente por WhatsApp** (requeriría integración con WhatsApp Business API que es un proyecto aparte). En cambio:

- El recepcionista hace clic en “Generar link” desde la reserva.
- El sistema genera el token y muestra el link completo.
- Botón “Copiar link” que copia al portapapeles.
- Botón “Abrir WhatsApp” que abre `https://wa.me/[número del huésped]?text=[mensaje predefinido con el link]`.

#### Mensaje predefinido sugerido:

Hola [Nombre], te enviamos el link para completar tu check-in online en Maran Sui

[LINK]

Solo te tomará 2 minutos. Cualquier consulta, escribinos por acá. ¡Te esperamos!

---

### Notas para Replit

- Implementar en orden: **Parte 1 (notificaciones) → Parte 2 (chatbot) → Parte 3 (web check-in)**.
- La Parte 1 es la base — sin el sistema de notificaciones las otras dos partes no tienen dónde mostrar sus alertas.
- La página de web check-in ( `/web-checkin/:token` ) debe ser accesible sin autenticación. Verificar que el router de Wouter no requiera auth para esa ruta.
- Los endpoints públicos ( `/api/public/...` ) deben estar explícitamente excluidos del middleware de autenticación cuando se implemente en el futuro.

- La foto del documento enviada como base64 puede ser grande. Considerar comprimir la imagen en el frontend antes de enviarla (canvas resize a max 800px).
  - El token del web check-in debe ser suficientemente largo e impredecible (usar `crypto.randomUUID()` o similar).
  - La clave secreta del webhook del chatbot ( `X-Chatbot-Secret` ) debe guardarse como variable de entorno, no hardcodeada en el código.
  - Mantener la nomenclatura existente (camelCase en TypeScript, snake\_case en SQL).
- 

*Documento generado para el proyecto Maran Suite System — Integraciones diferenciadoras*  
*Chatbot + Web Check-in*